ShopAgent

Agentic AI Customer Support System
Project Details & Technical Documentation

Project Links

Live Demo (AWS): https://akkp7m0s45.execute-api.us-east-1.amazonaws.com/prod/
GitHub Repository: https://github.com/Insane-HK/shopagent-agentic-ai-assignment

1. Project Overview

ShopAgent is an AI-powered customer support chatbot built for an online e-commerce store. It uses Amazon Bedrock's Nova Pro model to autonomously handle customer queries about orders, products, and store categories. The agent does not rely on hardcoded if/else routing. Instead, it uses an agentic tool-use loop where the LLM decides which backend tools to call, reads their results, and formulates natural, human-friendly responses.

The application is deployed as a fully serverless stack on AWS (Lambda + API Gateway), and features a modern chat UI with dark mode, persistent conversation threads, collapsible sidebar, and full tool execution tracing.

2. Project Approach

2.1 Problem Analysis

The core challenge was building an agent that can answer diverse customer queries without hardcoding responses. Rather than stuffing all product and order data into the system prompt (which causes context overflow and hallucination), I designed a tool-use architecture where the model receives zero data upfront and instead calls backend functions on demand.

2.2 Agentic Tool-Use Loop

The agent operates in a loop (max 6 iterations):

- Send user message + tool schemas to Amazon Bedrock Converse API
- If the model returns tool_use blocks, execute each tool locally (Python functions)
- Send tool results back to the model as toolResult messages
- Repeat until the model returns end_turn or the round limit is reached
- Clean the final response (strip any leaked thinking tags) and return to the frontend

This enables tool chaining: the model can call get_order() to retrieve an order, extract the product_id from the result, and automatically follow up with get_product() for more details -- all without being explicitly programmed to do so.

2.3 Available Tools

- **get_order(order_id)** -- Fetches order details (status, tracking, items) by order ID
- **search_products(query)** -- Keyword search across the product catalog
- **get_product(product_id)** -- Retrieves full product details by product ID

- **get_categories()** -- Lists all store categories with product counts and price ranges

2.4 Guardrails & Safety

The system prompt includes explicit guardrails that restrict the agent to customer support topics only. If a user asks off-topic questions (politics, general knowledge, etc.), the agent politely declines and redirects the conversation to store-related queries. This prevents misuse of the underlying LLM's general knowledge capabilities.

2.5 Frontend Design

The UI was built with vanilla HTML, CSS, and JavaScript -- no React or build tooling. Key features include:

- Left/right chat bubble layout (agent on left, user on right)
- Persistent chat threads stored in browser localStorage
- Dark mode toggle with preference persistence
- Collapsible sidebar with conversation history
- Expandable tool trace panel showing what the AI called and why
- Suggestion chips for common queries
- Responsive design for mobile and desktop

2.6 Deployment Strategy

The entire infrastructure is defined as code using AWS SAM/CloudFormation (template.yaml). Deployment is a two-command process: python build.py packages the Lambda zip, and python deploy.py uploads it to S3 and updates the CloudFormation stack. This ensures reproducible, version-controlled deployments with no manual AWS console interaction.

3. Tech Stack & Justification

Amazon Nova Pro on Bedrock (LLM)

Reason: Nova Pro provides native tool-use support through the Converse API with first-class toolUse and toolResult message types. Unlike prompt-engineering approaches to simulate function calling, this is a structured, reliable mechanism. Bedrock also uses IAM roles for authentication (no API keys to manage) and offers pay-per-token pricing, making it cost-effective for a demo that sits idle most of the time.

FastAPI + Python (Backend)

Reason: FastAPI is async-first, provides automatic request validation via Pydantic models, and auto-generates OpenAPI documentation. It is the modern standard for Python web APIs. Combined with Mangum (a single-line ASGI adapter), the exact same FastAPI application runs locally with uvicorn during development and inside AWS Lambda in production -- with zero code changes between the two environments.

Mangum (Lambda Adapter)

Reason: Mangum converts AWS Lambda/API Gateway events into standard ASGI requests that FastAPI can process. The entire Lambda handler file is three lines of code. This eliminates the need for separate Lambda-specific request handling logic.

Vanilla HTML/CSS/JavaScript (Frontend)

Reason: For a chat interface, React or similar frameworks would add unnecessary complexity -- a build step,

node_modules, and deployment pipeline -- with no meaningful UX benefit. Vanilla CSS with custom properties (--variables) powers the entire theme system; dark mode is a single CSS class that overrides these variables. localStorage provides persistent chat threads without needing a session database. Zero build step means the files are served directly by FastAPI's StaticFiles middleware.

AWS Lambda + API Gateway (Hosting)

Reason: Serverless architecture means $0 cost at idle, automatic scaling to handle traffic spikes, and zero server management. The first 1 million Lambda requests per month are free under the AWS Free Tier. API Gateway provides HTTPS endpoints, request routing, and throttling out of the box.

AWS SAM / CloudFormation (Infrastructure)

Reason: Infrastructure as Code ensures the deployment is reproducible and version-controlled. The template.yaml file defines the Lambda function, API Gateway, IAM roles, and S3 bucket. Changes are deployed atomically via CloudFormation stack updates -- if anything fails, it automatically rolls back.

Custom Agent Loop (over LangChain)

Reason: LangChain was deliberately avoided. It would add approximately 50MB of dependencies and abstract away the exact mechanism the assignment aims to demonstrate -- how an agentic tool-use loop works. The custom loop in agent.py is approximately 80 lines of Python and provides full control over error handling, tool tracing, response cleaning, and the conversation flow. Every tool call is recorded with its input and output, which directly powers the tool trace panel in the UI.

4. Key Design Decisions

- **Tool-use over prompt stuffing:** Data stays in Python functions; the model fetches only what it needs. This scales to real databases and avoids hallucination.
- **Errors as data, not exceptions:** When an order is not found, the tool returns {"error": "Order not found"} rather than raising an exception. The model reads this and responds naturally.
- **localStorage for chat persistence:** Avoids the need for a server-side session database while still providing a seamless multi-conversation experience.
- **Cache-busted static assets:** ?v=N query params on CSS/JS URLs force browsers to load the latest version after every deployment.
- **System prompt guardrails:** Explicit instructions prevent the agent from answering off-topic questions, keeping it focused on its customer support role.

5. Testing

12 unit tests (pytest) cover all tool functions: valid lookups, invalid IDs, case-insensitive matching, keyword search with multi-word queries, empty results, and category listing. Tests run with: python -m pytest tests/ -v

6. Future Improvements

- Streaming responses via Bedrock's converse_stream API + Server-Sent Events
- Replace mock data with DynamoDB for production-grade persistence
- Add authentication via API Gateway + Cognito

- Server-side chat history storage for cross-device access
- Step-by-step tool trace animation in the UI